

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Depuración y Pruebas de Software				
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN	22/04/2026	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): Quispe Flores Marco Ramiro Quispe Madariaga Jeferson Jofre Ramirez Ccahuana Max Edu Valdiviezo Tovar Alexander				NOTA:	
DOCENTE(s): Prof. Robert Arisaca					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Calcular el área de un rectángulo Enunciado: Escribe un programa que solicite al usuario ingresar la base y la altura de un rectángulo como números enteros o decimales. El programa debe calcular el área del rectángulo ($\text{Área} = \text{base} * \text{altura}$) e imprimir el resultado mostrando claramente cuál es la base, la altura y el área calculada. Objetivo de prueba: Verificar que el programa calcula correctamente el área para diferentes valores de base y altura (enteros, decimales, positivos). El informe debe contener el código fuente: • rectangle.py • rectangle_test.py rectangle.py

Python

```
def calcularArea(base, altura):
    """
    Esta función calcula el área de un rectángulo.
    Args:
    base: La longitud de la base del rectángulo.
    altura: La longitud de la altura del rectángulo.
    Returns:
    El área del rectángulo.
    """
    return base * altura

# Solicitar al usuario la longitud de los lados
base = int(input("Ingrese la longitud de la base: "))
altura = int(input("Ingrese la longitud de la altura: "))

# Calcular el área del rectángulo
area = calcularArea(base, altura)
print(f"El área del rectángulo es: {area}")
```

Procedemos a ingresar valores inválidos que hagan el programa fallar:

Valores numéricos problemáticos:

```
PS D:\PruSof\01> python .\rectangle.py
Ingrese la longitud de la base: 2
Ingrese la longitud de la altura: -2
El área del rectángulo es: -4
PS D:\PruSof\01>
```

```
PS D:\PruSof\01> python .\rectangle.py
Ingrese la longitud de la base: 0
Ingrese la longitud de la altura: 10
El área del rectángulo es: 0
PS D:\PruSof\01>
```

Valores no numéricos:

```
PS D:\PruSof\01> python .\rectangle.py
Ingrese la longitud de la base: a
Traceback (most recent call last):
  File "D:\PruSof\01\rectangle.py", line 13, in <module>
    base = int(input("Ingrese la longitud de la base: "))
ValueError: invalid literal for int() with base 10: 'a'
PS D:\PruSof\01>
```

```
PS D:\PruSof\01> python .\rectangle.py
Ingrese la longitud de la base:
Traceback (most recent call last):
  File "D:\PruSof\01\rectangle.py", line 13, in <module>
    base = int(input("Ingrese la longitud de la base: "))
ValueError: invalid literal for int() with base 10: ''
PS D:\PruSof\01>
```

En base a estos, creamos casos de prueba:

ID	Descripción	Datos Entrada	Salida esperada
1	Rectángulo válido (b>h)	(10,5)	50
2	Rectángulo válido (b<h)	(4,9)	36
3	Rectángulo válido (b=h)	(8,8)	64

4	Base negativa	(-5,10)	Rectángulo inválido: Las longitudes de los lados deben ser positivas.
5	Altura negativa	(10,-5)	Rectángulo inválido: Las longitudes de los lados deben ser positivas.
6	Base y altura negativas	(-4,-4)	Rectángulo inválido: Las longitudes de los lados deben ser positivas.
7	Base igual a cero	(0,10)	Rectángulo inválido: Las longitudes de los lados deben ser positivas.
8	Altura igual a cero	(10,0)	Rectángulo inválido: Las longitudes de los lados deben ser positivas.
9	Base y altura iguales a cero	(0,0)	Rectángulo inválido: Las longitudes de los lados deben ser positivas.
10	Valor no numérico en base	(a,10)	Ingrese solo valores numéricos válidos
11	Valor no numérico en altura	(10,b)	Ingrese solo valores numéricos válidos
12	Valor decimal en base	(3.5,10)	35
13	Valor decimal en altura	(10,4.8)	48
14	Entrada vacía en base	(,10)	Ingrese solo valores numéricos válidos
15	Entrada vacía en altura	(10,)	Ingrese solo valores numéricos válidos
16	Altura con caracteres especiales	(10,"4@")	Ingrese solo valores numéricos válidos
17	Más de dos valores ingresados	(10,20,30)	200 (El programa termina antes de poder ingresar un tercer valor)
18	Entrada con separador no estándar	("10,5")	Ingrese solo valores numéricos válidos

rectangle_test.py

Python

"""

Este programa lee dos números (enteros o decimales) que representan la base y la altura de un rectángulo y calcula su área, validando que los valores sean positivos.

"""

```
def calcularArea(base, altura):  
    """  
    Calcula el área de un rectángulo.  
    Args:  
    base: Longitud de la base (número positivo).  
    altura: Longitud de la altura (número positivo).  
    Returns:  
    El área del rectángulo.  
    """  
    return base * altura  
  
def validar_rectangulo(base, altura):  
    """  
    Valida que la base y la altura sean positivas.  
    Args:  
    base: Longitud de la base.  
    altura: Longitud de la altura.  
    Returns:  
    Una cadena de texto indicando si es válido o el error.  
    """  
    # Verificar que la base y altura sean positivas  
    if base <= 0 or altura <= 0:  
        return "Rectángulo inválido: Las longitudes de los lados deben ser positivas."  
    return "Válido"  
  
# --- Programa Principal ---  
if __name__ == "__main__":  
    print("Ingrese las longitudes de la base y la altura del rectángulo:")  
    try:  
        # Leer las entradas del usuario y convertirlas a float (permite enteros y  
        # decimales)  
        base = float(input("Base: "))  
        altura = float(input("Altura: "))  
  
        # Validar que los valores sean positivos  
        validacion = validar_rectangulo(base, altura)  
        if validacion != "Válido":  
            print(validacion)  
        else:  
            # Llamar a la función para calcular el área  
            area = calcularArea(base, altura)  
            # Mostrar el resultado (si es entero, mostrarlo sin decimales)  
            if area.is_integer():  
                print(f"El área del rectángulo es: {int(area)}")  
            else:  
                print(f"El área del rectángulo es: {area}")  
  
    except ValueError:
```

```
# Manejar el caso en que la entrada no sea un número válido
print("Error: Por favor, ingrese solo valores numéricos válidos.")
```

Ejecución:

```
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: 5
El área del rectángulo es: 50
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 4
Altura: 9
El área del rectángulo es: 36
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 8
Altura: 8
El área del rectángulo es: 64
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: -5
Altura: 10
Rectángulo inválido: Las longitudes de los lados deben ser positivas.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: -5
Rectángulo inválido: Las longitudes de los lados deben ser positivas.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: -4
Altura: -4
Rectángulo inválido: Las longitudes de los lados deben ser positivas.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 0
Altura: 10
Rectángulo inválido: Las longitudes de los lados deben ser positivas.
```

```
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: 0
Rectángulo inválido: Las longitudes de los lados deben ser positivas.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 0
Altura: 0
Rectángulo inválido: Las longitudes de los lados deben ser positivas.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: a
Error: Por favor, ingrese solo valores numéricos válidos para las longitudes de los lados.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: b
Error: Por favor, ingrese solo valores numéricos válidos para las longitudes de los lados.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: a
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: b
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 3.5
Altura: 10
El área del rectángulo es: 35
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: 4.8
El área del rectángulo es: 48
```

```
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base:
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura:
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: 4@
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 9999999999
Altura: 9999999999
El área del rectángulo es: 99999999980000002048
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10
Altura: 20 30
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> python .\rectangle_test.py
Ingrese las longitudes de la base y la altura del rectángulo:
Base: 10, 5
Error: Por favor, ingrese solo valores numéricos válidos.
PS D:\PruSof\01> 
```

2. Identificador de Números Pares e Impares en una Lista

Enunciado:

Crea un programa que primero solicite al usuario cuántos números desea ingresar. Luego, debe pedirle que ingrese cada uno de esos números enteros. Finalmente, el programa debe recorrer la lista de números ingresados e imprimir cada número junto con un mensaje indicando si es "par" o "impar".

Objetivo de prueba: Comprobar que el programa maneja correctamente la cantidad de números especificada, lee los números, e identifica y etiqueta correctamente cada número como par o impar (incluyendo el número cero y números negativos si se consideran).

El informe debe contener el código fuente:

- numbers.py
- numbers_test.py

numbers.py

Python

```
def par_impar(nums):  
    """  
    En esta función se clasifica en par o impar cada número n  
    de la lista nums, se devuelve la lista resultado con tuplas (int,String)  
    """  
    resultado = []  
    for n in nums:  
        if n % 2 == 0:  
            resultado.append((n, "par"))  
        else:  
            resultado.append((n, "impar"))  
    return resultado  
  
def verifCantidad(entrada):  
    """  
    En esta función se verifica que la cantidad de números a clasificar  
    corresponda a un entero.  
    """  
    try:  
        cantidad = int(entrada)  
        if cantidad <= 0:  
            return "Cantidad inválida, debe ingresar un número entero mayor que cero"  
        return cantidad  
    except ValueError:  
        return "Cantidad inválida, debe ingresar un número entero"  
  
def verifNumero(entrada):  
    """
```

En esta función se verifica que los números ingresados son enteros.

"""

try:

 return int(entrada)

except ValueError:

 return "Número inválido, solo se permiten números enteros"

"""

Desarrollo del programa

Se hacen uso de los verificadores y se hace uso de isinstance para saber si lo recibido corresponde a un error e imprimirlo

"""

if __name__ == "__main__":

 nums = []

 entrada = input("Ingrese la cantidad de números a clasificar: ")

 cantidad = verifCantidad(entrada)

 if isinstance(cantidad, str):

 print(cantidad)

 exit()

 for i in range(cantidad):

 entrada = input("Ingrese número: ")

 numero = verifNumero(entrada)

 if isinstance(numero, str):

 print(numero)

 exit()

 nums.append(numero)

 procesados = par_impar(nums)

 for numero, tipo in procesados:

 print(numero, " es ", tipo)

Ejecución

```
LabPS/numbers.py
Ingrese la cantidad de números a clasificar: 4
Ingrese número: 1
Ingrese número: -2
Ingrese número: -3
Ingrese número: 4
1 es impar
-2 es par
-3 es impar
4 es par
```

numbers_test.py

Python

```
from numbers import par_impar, verifCantidad, verifNumero

def test_par_impar():
    """
    Test para la clasificación correcta de números como pares o impares.
    """

    print("\nTest clasificación de par impar en los números (1, -2 y 3)")

    esperado = [(1, "impar"), (-2, "par"), (3, "impar")]
    print("Resultado esperado: ", esperado)

    resultado = par_impar([1, -2, 3])
    print("Resultado obtenido:", resultado)

    assert resultado == esperado
    print("Test de clasificación par o impar correcto")

def test_verifCantidad():
    """
    Test para la verificación de la cantidad de números ingresados.
    Casos válidos e inválidos
    """

    print("\nTest cantidad válida (5):")
    print("Resultado:", verifCantidad("5"))
    assert verifCantidad("5") == 5

    print("\nTest cantidad inválida (0):")
    print("Resultado:", verifCantidad("0"))
    assert "Cantidad inválida" in verifCantidad("0")

    print("\nTest cantidad inválida (-3):")
    print("Resultado:", verifCantidad("-3"))
    assert "Cantidad inválida" in verifCantidad("-3")

    print("\nTest cantidad inválida (texto 'abc'):")
    print("Resultado:", verifCantidad("abc"))
    assert "Cantidad inválida" in verifCantidad("abc")

    print("Test de validación de cantidad correcto")

def test_verifNumero():
    """
```

Test para la verificación de números que son ingresados.
Casos válidos e inválidos.
"""

```
print("\nTest número válido (10):")
print("Resultado:", verifNumero("10"))
assert verifNumero("10") == 10

print("\nTest número válido (-7):")
print("Resultado:", verifNumero("-7"))
assert verifNumero("-7") == -7

print("\nTest número inválido (decimal 2.5):")
print("Resultado:", verifNumero("2.5"))
assert "Número inválido" in verifNumero("2.5")

print("\nTest número inválido (texto 'abc'):")
print("Resultado:", verifNumero("abc"))
assert "Número inválido" in verifNumero("abc")

print("Test de validación de números correcto")

if __name__ == "__main__":
    test_par_impar()
    test_verifCantidad()
    test_verifNumero()

    print("\nTodas las pruebas pasaron correctamente")
```

Ejecución

```
LabPS/numbers_test.py

Test clasificación de par impar en los números (1, -2 y 3)
Resultado esperado: [(1, 'impar'), (-2, 'par'), (3, 'impar')]
Resultado obtenido: [(1, 'impar'), (-2, 'par'), (3, 'impar')]
Test de clasificación par o impar correcto

Test cantidad válida (5):
Resultado: 5

Test cantidad inválida (0):
Resultado: Cantidad inválida, debe ingresar un número entero mayor que cero

Test cantidad inválida (-3):
Resultado: Cantidad inválida, debe ingresar un número entero mayor que cero

Test cantidad inválida (texto 'abc'):
Resultado: Cantidad inválida, debe ingresar un número entero
Test de validación de cantidad correcto

Test número válido (10):
Resultado: 10

Test número válido (-7):
Resultado: -7

Test número inválido (decimal 2.5):
Resultado: Número inválido, solo se permiten números enteros

Test número inválido (texto 'abc'):
Resultado: Número inválido, solo se permiten números enteros
Test de validación de números correcto

Todas las pruebas pasaron correctamente
```

Pruebas realizadas manualmente.

Pruebas de ingreso de cantidad

ID	Descripción	Datos entrada	Salida esperada
c1	Ingreso válido de cantidad	5	Se acepta el valor
c2	Ingreso de cero	0	Cantidad inválida, debe ingresar un número entero mayor que cero
c3	Ingreso de número negativo	-3	Cantidad inválida, debe ingresar un número entero mayor que cero
c4	Ingreso de letra	a	Cantidad inválida, debe ingresar un número entero
c5	Ingreso de texto	hola	Cantidad inválida, debe ingresar un número entero

Pruebas de ingreso de números

ID	Descripción	Datos entrada	Salida esperada
n1	Número entero positivo	10	Se acepta el número
n2	Número entero negativo	-7	Se acepta el número
n3	Número decimal	2.5	Número inválido, solo se permiten números enteros
n4	Letra	q	Número inválido, solo se permiten números enteros
n5	Texto	hola	Número inválido, solo se permiten números enteros

Pruebas de procesamiento (par/impar)

ID	Descripción	Datos entrada	Salida esperada
p1	Número par	2	2 es par
p2	Número impar	3	3 es impar
p3	Número cero	0	0 es par
p4	Número negativo par	-4	-4 es par
p5	Número negativo impar	-5	-5 es impar

3. Mini Simulador de Cajero Automático (ATM)

Enunciado:

Desarrolla un programa que simule las operaciones básicas de un cajero automático. El programa debe iniciar con un saldo fijo (por ejemplo, S/.1,000). Debe presentar un menú al usuario con las siguientes opciones:

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

El programa debe permitir al usuario seleccionar una opción.

- Si elige Consultar Saldo, debe mostrar el saldo actual.
- Si elige Depositar Dinero, debe solicitar la cantidad a depositar (validar que sea un monto positivo) y actualizar el saldo.
- Si elige Retirar Dinero, debe solicitar la cantidad a retirar (validar que sea un monto positivo y que no

exceda el saldo disponible) y actualizar el saldo.

- Si elige Salir, el programa debe terminar.

El menú debe mostrarse repetidamente hasta que el usuario elija la opción "Salir".

Objetivo de prueba: Verificar que todas las opciones del menú funcionan correctamente: la consulta muestra el saldo correcto, los depósitos incrementan el saldo adecuadamente, los retiros disminuyen el saldo (respetando el límite del saldo disponible y validando montos positivos), y la opción de salir termina el programa. Probar casos límite como intentar retirar más dinero del disponible o depositar/retirar cero o montos negativos.

El informe debe contener el código fuente:

- atm.py
- atm_test.py

atm.py

Python

```
"""
Este programa simula las operaciones básicas de un cajero automático.
Inicia con un saldo fijo de S/.1000 y permite consultar saldo, depositar,
retirar dinero o salir.
"""

def consultar_saldo(saldo):
    """
    Muestra el saldo actual.
    Args:
    saldo: El saldo actual de la cuenta.
    Returns:
    El saldo actual (sin cambios).
    """
    return saldo

def depositar(saldo, monto):
    """
    Actualiza el saldo sumando el monto depositado.
    Args:
    saldo: El saldo actual de la cuenta.
    monto: La cantidad a depositar.
    Returns:
    El nuevo saldo actualizado.
    """
    return saldo + monto

def retirar(saldo, monto):
    """
    Actualiza el saldo restando el monto retirado.
    Args:
```

```
saldo: El saldo actual de la cuenta.  
monto: La cantidad a retirar.  
Returns:  
El nuevo saldo actualizado.  
""  
  
return saldo - monto
```

```
# --- Programa Principal ---  
if __name__ == "__main__":  
    saldo = 1000 # Saldo inicial fijo  
  
    while True:  
        print("\n--- CAJERO AUTOMÁTICO ---")  
        print("1. Consultar Saldo")  
        print("2. Depositar Dinero")  
        print("3. Retirar Dinero")  
        print("4. Salir")  
  
        opcion = int(input("Seleccione una opción: "))  
  
        if opcion == 1:  
            # Consultar saldo  
            saldo_actual = consultar_saldo(saldo)  
            print(f"Su saldo actual es: S/.{saldo_actual}")  
  
        elif opcion == 2:  
            # Depositar dinero  
            monto = int(input("Ingrese la cantidad a depositar: S/."))  
            saldo = depositar(saldo, monto)  
            print(f"Depósito exitoso. Su nuevo saldo es: S/.{saldo}")  
  
        elif opcion == 3:  
            # Retirar dinero  
            monto = int(input("Ingrese la cantidad a retirar: S/."))  
            if monto <= saldo:  
                saldo = retirar(saldo, monto)  
                print(f"Retiro exitoso. Su nuevo saldo es: S/.{saldo}")  
            else:  
                print("Error: Saldo insuficiente.")  
  
        elif opcion == 4:  
            # Salir  
            print("Gracias por usar el cajero automático. ¡Hasta luego!")  
            break  
  
        else:  
            print("Opción no válida. Por favor, seleccione una opción del 1 al 4.")
```

Ejecución:

```
PS D:\PruSof\01> python .\atm.py
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 1
```

```
Su saldo actual es: S/.1000
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 2
```

```
Ingrese la cantidad a depositar: S/.200
```

```
Depósito exitoso. Su nuevo saldo es: S/.1200
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 3
```

```
Ingrese la cantidad a retirar: S/.300
```

```
Retiro exitoso. Su nuevo saldo es: S/.900
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 4
```

```
Gracias por usar el cajero automático. ¡Hasta luego!
```

```
PS D:\PruSof\01> python .\atm.py
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 1
```

```
Su saldo actual es: S/.1000
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 2
```

```
Ingrese la cantidad a depositar: S/.-300
```

```
Depósito exitoso. Su nuevo saldo es: S/.700
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: 3
```

```
Ingrese la cantidad a retirar: S/.2000
```

```
Error: Saldo insuficiente.
```

```
--- CAJERO AUTOMÁTICO ---
```

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

```
Seleccione una opción: a
```

```
Traceback (most recent call last):
```

```
File "D:\PruSof\01\atm.py", line 53, in <module>
```

```
opcion = int(input("Seleccione una opción: "))
```

```
ValueError: invalid literal for int() with base 10: 'a'
```

```
PS D:\PruSof\01>
```

Definimos los casos de prueba:

ID	Descripción	Opción	Monto	Salida esperada
1	Consultar saldo inicial	1	N/A	Su saldo actual es: S/.1000
2	Depositar monto válido positivo	2	500	Depósito exitoso. Su nuevo saldo es: S/.1500
3	Depositar monto con decimales	2	250.75	Depósito exitoso. Su nuevo saldo es: S/.1250.75
4	Depositar monto cero	2	0	Error: El monto a depositar debe ser mayor a cero.
5	Depositar monto negativo	2	-200	Error: El monto a depositar

				debe ser mayor a cero.
6	Depositar valor no numérico	2	abc	Error: Por favor ingrese un monto válido
7	Depositar entrada vacía	2	(Enter)	Error: Por favor ingrese un monto válido
8	Retirar monto válido (menor al saldo)	3	300	Retiro exitoso. Su nuevo saldo es: S/.700
9	Retirar monto válido (igual al saldo)	3	1000	Retiro exitoso. Su nuevo saldo es: S/.0
10	Retirar monto con decimales	3	125.50	Retiro exitoso. Su nuevo saldo es: S/_.874.50
11	Retirar monto mayor al saldo	3	1500	Error: Saldo insuficiente. Su saldo actual es: S/_.1000
12	Retirar monto cero	3	0	Error: El monto a retirar debe ser mayor que cero.
13	Retirar monto negativo	3	-100	Error: El monto a retirar debe ser mayor que cero.
14	Retirar valor no numérico	3	xyz	Error: Por favor ingrese un monto válido
15	Retirar entrada vacía	3	(Enter)	Error: Por favor ingrese un monto válido
16	Seleccionar opción no numérica	a	N/A	Error: Por favor, seleccione una opción válida (1-4).
17	Seleccionar opción fuera de rango (0)	0	N/A	Opción no válida. Por favor, seleccione una opción del 1 al 4.
18	Seleccionar opción fuera de rango (5)	5	N/A	Opción no válida. Por favor, seleccione una opción del 1 al 4.
19	Salir del programa	4	N/A	Gracias por usar el cajero automático. ¡Hasta luego! (El programa termina)
20	Depositar, luego retirar, luego	2, 3, 1	Depositar 500,	Depósito: S/_.1500, Retiro:

	consultar		retirar 200	S/.1300, Consulta: "Su saldo actual es: S/.1300"
21	Depositar monto muy grande	2	9999999999	Depósito exitoso. Su nuevo saldo es: S/.10000000099
22	Retirar monto muy grande	3	9999999999	Retiro exitoso. Su nuevo saldo es: S/.1000
23	Ingresar opción con espacios	"2 "	N/A	(Opción 2)
24	Depositar monto con símbolos	2	\$500	Error: Por favor, ingrese un monto válido.
25	Retirar monto justo después de depositar	2, 3, 1	Depositar 300, retirar 1400	Depósito: S/.1300, Retiro falla por saldo insuficiente

atm_test.py

Python

```

"""
Este programa simula las operaciones básicas de un cajero automático.
Inicia con un saldo fijo de S/.1000 y permite consultar saldo, depositar,
retirar dinero o salir.
Valida que los montos sean números positivos, maneja decimales,
y verifica saldo suficiente antes de retirar.
"""

def consultar_saldo(saldo):
    """
    Muestra el saldo actual.
    Args:
    saldo: El saldo actual de la cuenta.
    Returns:
    El saldo actual (sin cambios).
    """
    return saldo

def depositar(saldo, monto):
    """
    Actualiza el saldo sumando el monto depositado.
    Args:
    saldo: El saldo actual de la cuenta.
    monto: La cantidad a depositar (debe ser positivo).
    Returns:
    El nuevo saldo actualizado.
    """

```

```
return saldo + monto
```

```
def retirar(saldo, monto):
```

```
    """
```

```
    Actualiza el saldo restando el monto retirado.
```

```
    Args:
```

```
    saldo: El saldo actual de la cuenta.
```

```
    monto: La cantidad a retirar (debe ser positivo y menor o igual al saldo).
```

```
    Returns:
```

```
    El nuevo saldo actualizado.
```

```
    """
```

```
    return saldo - monto
```

```
def obtener_monto_positivo(mensaje):
```

```
    """
```

```
    Solicita un monto positivo al usuario, validando entradas.
```

```
    Maneja valores no numéricos, decimales, vacíos, negativos y cero.
```

```
    Args:
```

```
    mensaje: El mensaje a mostrar al usuario.
```

```
    Returns:
```

```
    El monto válido (float).
```

```
    """
```

```
    while True:
```

```
        try:
```

```
            entrada = input(mensaje).strip()
```

```
            # Validar entrada vacía
```

```
            if not entrada:
```

```
                print("Error: Por favor, ingrese un monto válido.")
```

```
                continue
```

```
            # Intentar convertir a float (permite enteros y decimales)
```

```
            monto = float(entrada)
```

```
            # Validar que sea mayor que cero
```

```
            if monto <= 0:
```

```
                print("Error: El monto debe ser mayor a cero.")
```

```
                continue
```

```
            return monto
```

```
        except ValueError:
```

```
            print("Error: Por favor, ingrese un monto válido.")
```

```
def obtener_opcion_valida():
```

```
    """
```

```
    Solicita una opción válida del menú al usuario.
```

Maneja entradas no numéricas, vacías y fuera de rango.

Returns:

La opción válida (entero del 1 al 4).

"""

```
while True:
```

```
    try:
```

```
        entrada = input("Seleccione una opción: ").strip()
```

```
        # Validar entrada vacía
```

```
        if not entrada:
```

```
            print("Error: Por favor, seleccione una opción válida (1-4).")
```

```
            continue
```

```
        # Intentar convertir a entero
```

```
        opcion = int(entrada)
```

```
        # Validar que esté en el rango 1-4
```

```
        if opcion < 1 or opcion > 4:
```

```
            print("Opción no válida. Por favor, seleccione una opción del 1 al 4.")
```

```
            continue
```

```
        return opcion
```

```
    except ValueError:
```

```
        print("Error: Por favor, seleccione una opción válida (1-4).")
```

```
def formatear_saldo(saldo):
```

```
    """
```

```
    Formatea el saldo para mostrarlo correctamente.
```

```
    Si es entero, muestra sin decimales; si tiene decimales, muestra con 2 decimales.
```

```
    Args:
```

```
    saldo: El saldo a formatear.
```

```
    Returns:
```

```
    El saldo formateado como string.
```

```
    """
```

```
    if saldo.is_integer():
```

```
        return f"$/{int(saldo)}"
```

```
    else:
```

```
        return f"$/{saldo:.2f}"
```

```
# --- Programa Principal ---
```

```
if __name__ == "__main__":
```

```
    saldo = 1000.0 # Saldo inicial fijo
```

```
    print("Bienvenido al cajero automático.")
```

```
    while True:
```

```
        print("\n--- CAJERO AUTOMÁTICO ---")
```

```
print("1. Consultar Saldo")
print("2. Depositar Dinero")
print("3. Retirar Dinero")
print("4. Salir")

opcion = obtener_opcion_valida()

if opcion == 1:
    # Consultar saldo
    saldo_actual = consultar_saldo(saldo)
    print(f"Su saldo actual es: {formatear_saldo(saldo_actual)}")

elif opcion == 2:
    # Depositar dinero
    monto = obtener_monto_positivo("Ingrese la cantidad a depositar: S/.")
    saldo = depositar(saldo, monto)
    print(f"Depósito exitoso. Su nuevo saldo es: {formatear_saldo(saldo)}")

elif opcion == 3:
    # Retirar dinero
    monto = obtener_monto_positivo("Ingrese la cantidad a retirar: S/.")

    # Validar saldo suficiente
    if monto > saldo:
        print(f"Error: Saldo insuficiente. Su saldo actual es:
{formatear_saldo(saldo)}")
    else:
        saldo = retirar(saldo, monto)
        print(f"Retiro exitoso. Su nuevo saldo es: {formatear_saldo(saldo)}")

elif opcion == 4:
    # Salir
    print("Gracias por usar el cajero automático. ¡Hasta luego!")
    break
```

Ejecución:

```
PS D:\PruSof\01> python .\atm_test.py
Bienvenido al cajero automático.

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 1
Su saldo actual es: S/.1000

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 2
Ingrese la cantidad a depositar: S/.500
Depósito exitoso. Su nuevo saldo es: S/.1500
```

```
PS D:\PruSof\01> python .\atm_test.py
Bienvenido al cajero automático.

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 3
Ingrese la cantidad a retirar: S/.300
Retiro exitoso. Su nuevo saldo es: S/.700

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 3
Ingrese la cantidad a retirar: S/.700
Retiro exitoso. Su nuevo saldo es: S/.0
```

```
PS D:\PruSof\01> python .\atm_test.py
Bienvenido al cajero automático.

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 2
Ingrese la cantidad a depositar: S/.250.75
Depósito exitoso. Su nuevo saldo es: S/.1250.75

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 2
Ingrese la cantidad a depositar: S/.0
Error: El monto debe ser mayor a cero.
Ingrese la cantidad a depositar: S/-.200
Error: El monto debe ser mayor a cero.
Ingrese la cantidad a depositar: S/.abc
Error: Por favor, ingrese un monto válido.
Ingrese la cantidad a depositar: S/.
Error: Por favor, ingrese un monto válido.
Ingrese la cantidad a depositar: S/.
```

```
PS D:\PruSof\01> python .\atm_test.py
Bienvenido al cajero automático.

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 3
Ingrese la cantidad a retirar: S/.125.50
Retiro exitoso. Su nuevo saldo es: S/.874.50

--- CAJERO AUTOMÁTICO ---
1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir
Seleccione una opción: 3
Ingrese la cantidad a retirar: S/.1500
Error: Saldo insuficiente. Su saldo actual es: S/.874.50
```

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 3

Ingrese la cantidad a retirar: S/.0

Error: El monto debe ser mayor a cero.

Ingrese la cantidad a retirar: S/-.100

Error: El monto debe ser mayor a cero.

Ingrese la cantidad a retirar: S/.xyz

Error: Por favor, ingrese un monto válido.

Ingrese la cantidad a retirar: S/.

Error: Por favor, ingrese un monto válido.

Ingrese la cantidad a retirar: S/.125.50

Retiro exitoso. Su nuevo saldo es: S/.749

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: a

Error: Por favor, seleccione una opción válida (1-4).

Seleccione una opción: 0

Opción no válida. Por favor, seleccione una opción del 1 al 4.

Seleccione una opción: 5

Opción no válida. Por favor, seleccione una opción del 1 al 4.

Seleccione una opción: 4

Gracias por usar el cajero automático. ¡Hasta luego!

PS D:\PruSof\01> python .\atm_test.py

Bienvenido al cajero automático.

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 2

Ingrese la cantidad a depositar: S/.500

Depósito exitoso. Su nuevo saldo es: S/.1500

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 3

Ingrese la cantidad a retirar: S/.200

Retiro exitoso. Su nuevo saldo es: S/.1300

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 1

Su saldo actual es: S/.1300

PS D:\PruSof\01> python .\atm_test.py

Bienvenido al cajero automático.

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 2

Ingrese la cantidad a depositar: S/.9999999999

Depósito exitoso. Su nuevo saldo es: S/.10000000999

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 3

Ingrese la cantidad a retirar: S/.9999999999

Retiro exitoso. Su nuevo saldo es: S/.1000

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 2

Ingrese la cantidad a depositar: S/.\$500

Error: Por favor, ingrese un monto válido.

Ingrese la cantidad a depositar: S/.300

Depósito exitoso. Su nuevo saldo es: S/.1300

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 3

Ingrese la cantidad a retirar: S/.400

Retiro exitoso. Su nuevo saldo es: S/.900

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 1400

Opción no válida. Por favor, seleccione una opción del 1 al 4.

Seleccione una opción: 3

Ingrese la cantidad a retirar: S/.1400

Error: Saldo insuficiente. Su saldo actual es: S/.900

--- CAJERO AUTOMÁTICO ---

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

Seleccione una opción: 4

Gracias por usar el cajero automático. ¡Hasta luego!

PS D:\PruSof\01> □

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 23

II. CUESTIONARIO

1. ¿Cuál es el objetivo de la depuración de software?

El objetivo de la depuración de software es identificar, analizar y corregir los errores o fallos que se presentan en un programa, con la finalidad de comprender por qué no funciona correctamente y solucionar dichos problemas. Este proceso permite encontrar la causa de los errores en el código y eliminarlos para asegurar un funcionamiento adecuado del sistema (1).

Además, la depuración contribuye a mejorar la calidad, el rendimiento y la confiabilidad del software, ya que ayuda a prevenir fallos futuros y optimizar su ejecución. Al corregir los errores detectados, se logra un sistema más estable y eficiente, lo que favorece una mejor experiencia para el usuario final (2).

2. ¿Cuál es el objetivo de las pruebas de software?

El objetivo de las pruebas de software es verificar y validar que un sistema o aplicación funcione conforme a los requisitos establecidos, identificando posibles errores o defectos antes de su implementación. Este proceso permite evaluar si el software cumple con las especificaciones definidas y si responde correctamente a diferentes condiciones de uso (3).

Asimismo, las pruebas buscan asegurar la calidad, confiabilidad y rendimiento del software, reduciendo riesgos y garantizando una experiencia adecuada para el usuario. A través de distintos tipos de pruebas, se puede detectar fallos de manera temprana, lo que facilita su corrección y contribuye a un producto final más estable y eficiente (4).

3. Investigue sobre el perfil que debería tener un analista de pruebas y un ingeniero de pruebas de software

El analista de pruebas de software debe contar con habilidades analíticas y atención al detalle, ya que su principal función es diseñar, ejecutar y documentar casos de prueba para identificar defectos en el sistema. Además, necesita comprender los requisitos del software y tener conocimientos básicos de programación y metodologías de desarrollo, lo que le permite evaluar el comportamiento del sistema desde la perspectiva del usuario y asegurar que cumpla con los criterios establecidos (5).

Por otro lado, el ingeniero de pruebas de software suele tener un perfil más técnico, orientado a la automatización de pruebas y al uso de herramientas especializadas (6). Este profesional requiere conocimientos avanzados en lenguajes de programación, integración continua y frameworks de testing, lo que le permite desarrollar scripts automatizados, optimizar procesos de prueba y mejorar la eficiencia en la detección de errores dentro del ciclo de desarrollo.

En conjunto, ambos perfiles deben poseer habilidades blandas como la comunicación efectiva, el

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 24

trabajo en equipo y la capacidad de resolución de problemas, ya que interactúan constantemente con desarrolladores, analistas y otros miembros del equipo (7). Estas competencias, junto con una actualización constante en tecnologías y metodologías ágiles, son clave para garantizar la calidad del software y adaptarse a las demandas cambiantes del entorno tecnológico.

4. Dado que las pruebas exhaustivas (probar todas las combinaciones posibles de entradas y condiciones) son impracticables para la mayoría del software real. De su opinión sobre los criterios, técnicas a utilizar para garantizar una alta confiabilidad del software.

Dado que resulta imposible probar todas las combinaciones de entradas y condiciones en la mayoría de los sistemas reales, es necesario aplicar criterios de selección que permitan enfocar los esfuerzos en los casos más relevantes. Uno de los principales criterios es el análisis de riesgos, que prioriza aquellas funcionalidades críticas o con mayor probabilidad de fallo, asegurando que los recursos se utilicen donde más impacto pueden tener.

Asimismo, se emplean técnicas de diseño de pruebas como la partición de equivalencia y el análisis de valores límite, las cuales permiten reducir el número de casos de prueba sin perder efectividad. Estas técnicas se basan en seleccionar datos representativos que cubran diferentes escenarios posibles, aumentando la probabilidad de detectar errores sin necesidad de evaluar todas las combinaciones.

Por otro lado, el uso de pruebas automatizadas y la integración continua contribuyen significativamente a mejorar la confiabilidad del software. Estas prácticas permiten ejecutar pruebas de manera frecuente y consistente, detectando fallos de forma temprana y facilitando su corrección antes de que el sistema avance a etapas más críticas del desarrollo.

Finalmente, la combinación de pruebas funcionales y no funcionales, junto con revisiones de código y buenas prácticas de desarrollo, fortalece la calidad del producto final. Al aplicar un enfoque integral de aseguramiento de calidad, se logra un equilibrio entre esfuerzo y cobertura, incrementando la confiabilidad del software incluso sin realizar pruebas exhaustivas (8).

5. ¿Cuál es la diferencia entre verificación y validación de software?

La verificación de software se enfoca en comprobar que el producto se esté desarrollando correctamente de acuerdo con las especificaciones y requisitos definidos desde el inicio. Es un proceso interno que evalúa si cada fase del desarrollo cumple con los estándares establecidos, utilizando revisiones, inspecciones y pruebas para asegurar que el sistema se construya de manera adecuada (9).

En cambio, la validación de software busca asegurar que el producto final satisfaga las necesidades y expectativas del usuario. Este proceso se centra en evaluar si el sistema realmente resuelve el problema para el cual fue creado, mediante pruebas funcionales y escenarios reales de uso. En resumen, mientras la verificación responde a “¿estamos construyendo el producto correctamente?”, la validación responde a “¿estamos construyendo el producto correcto?” (10).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 25

III. CONCLUSIONES

- Se comprobó que en el ejercicio de cálculo de área del rectángulo, la incorporación de validaciones para entradas numéricas y valores positivos permitió evitar resultados incorrectos, evidenciando una mejora en la precisión de los cálculos obtenidos.
- Se verificó que en el programa de identificación de números pares e impares, la aplicación de funciones de validación y procesamiento permitió clasificar correctamente los datos ingresados, incluso en casos límite como números negativos y cero, asegurando consistencia en los resultados.
- Se evidenció que en el simulador de cajero automático, la implementación de controles sobre montos y opciones del usuario permitió gestionar adecuadamente operaciones críticas como depósitos y retiros, previniendo errores como saldo insuficiente o entradas inválidas.
- Se determinó que la aplicación conjunta de pruebas (manuales y automatizadas) en los tres ejercicios permitió identificar fallos en distintos escenarios, optimizando el comportamiento del software y garantizando resultados más confiables y alineados a los requisitos planteados.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- (1) Amazon Web Services (AWS). What is Debugging? Disponible en: <https://aws.amazon.com/es/what-is-debugging/>
- (2) IBM. ¿Qué es la depuración (debugging)? Disponible en: <https://www.ibm.com/mx-es/think/topics/debugging>
- (3) ISTQB. (s.f.). What is software testing? <https://www.istqb.org/>
- (4) IBM. (s.f.). Software testing. <https://www.ibm.com/topics/software-testing>
- (5) ISTQB. (s.f.). Certified Tester Foundation Level Syllabus. <https://www.istqb.org/>
- (6) Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3rd ed.). John Wiley & Sons.
- (7) Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed., pp. 459–520). McGraw-Hill.
- (8) Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed., pp. 459–480). McGraw-Hill.
- (9) Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed., pp. 418–430). McGraw-Hill.
- (10) Sommerville, I. (2016). Software engineering (10th ed.). Pearson.